SCHOOL OF COMPUTER SCIENCE · UPES DEHRADUN

SpeakQL

Backend Plan · Revision 5

Revision 5 closes the design gaps Revision 4 left open. It gives the member overlay a read path, stops a stale merge from overwriting a newer value, defines the confidence number that six decisions already depended on, states how an external database is registered safely, and collects every endpoint into one contract the frontend can be built against.

Planning document. No implementation code.

Minor Project · MCA (AI & ML)
Garv Bhargava · Dhruv Chhatrawal · Kritant Poudnel
Mentor: Mr. Pankaj Dadure

0. What Revision 5 changes

Every row is a decision taken since Revision 4, and what it costs. Revision 4 removed things that were not paying for themselves; Revision 5 adds the four mechanisms that were described by their behaviour and left without a design, plus the contract that lets the two halves of the project be built in parallel.

#	REVISION 4	REVISION 5	SECTION
1	A member's pending value "appears in their own answers". The mechanism was never given, and `speakql_ro` holds no grant on `member_edits`.	The executor resolves reads through a per-member overlay view that `COALESCE`s pending values over the real table. `speakql_ro` gains SELECT on those views and nothing else.	§10.4
2	A merge applied the member's value to "the row the request names".	A merge re-reads the live row and compares it with the before value the request was raised against. If it moved, nothing is written and the request goes stale. This was a silent data-loss bug.	§10.5
3	Confidence gated routing, clarification and the seal band. The backend never said what the number was.	Defined here: the length-normalised beam score, with where it is read and what it does not claim.	§11.3
4	"An external database connected by credentials" was a capability with no registration flow.	Host resolution and private-range refusal, required TLS, a test connection, and a check that the supplied account cannot write.	§9.3
5	Grants were checked per request. Object identity — a `message_id`, a `thread_id` — was implied.	Every identifier is resolved to its owning person and organisation before use, and a guessed one is indistinguishable from one that does not exist.	§6.1
6	Rate limits existed only on the login flow.	Limits on asking, uploading and exporting — the three paths that cost model time, disk or money.	§7.4
7	Ingestion registered a schema and never re-indexed it, so an uploaded table was not retrievable until somebody ran `make reindex`.	Reindex is the last stage of ingestion. The demonstration script depends on this.	§9.2
8	Four safety layers, none of which covered a warehouse value reaching the explainer.	Result rows are passed as delimited data, never as instructions, and the checkable part of an explanation comes from the registry rather than the model.	§11.4
9	Endpoints named in passing across five sections. §14 was a file inventory.	One API contract: every route, its request and response shape, the streaming envelope and the error codes.	§12
10	A free-mail address was refused outright, which also turned away the individual analyst, the consultant and the student.	It signs up and gets a **personal workspace** — an organisation with a null domain, which domain resolution can therefore never match. Two people on gmail.com get two workspaces, not one shared tenant.	§7.2
11	The escalation path was "a larger hosted model". The largest component in the system was a capability with no name, no size, and no stated limit on what it is allowed to decide.	Named. It is **Gemma** (`gemma3:4b` under Ollama), with its two jobs written down — generation below the confidence threshold, and the explainer — and the one thing it is never asked to do, which is decide whether a statement may run.	§11.5

Two things Revision 4 got right and Revision 5 keeps unchanged: editing in place rather than through a staging mirror, and one owner per domain decided by a question asked once. Nothing in this revision reintroduces a concept Revision 4 removed.

1. Backend at a glance

ASPECT	ANSWER
What it is	The service between "a question arrives" and "an answer, a chart, an explanation, or a refusal goes back" — bracketed by who may ask, what they may see, and whether they may correct what they find.
Who uses it	Two roles. An owner registered the company and runs it. A member is an employee who asks questions. Each database grant separately carries a database role of analyst or viewer.
How someone gets in	Passwordless. A 6-digit code verifies the email at signup, and a fresh code is issued at every sign-in.
Which role you get	The email domain decides. An unseen domain asks the person once whether they are setting the company up or joining it. A known domain never asks.
Where a database comes from	An external database connected by credentials and checked at registration (§9.3), or a CSV, Excel or TXT upload that becomes a real queryable table — in a new database or an existing one, as the uploader chooses.
What a member sees	Only the databases explicitly granted, at the depth their persona sets.
Local model vs fallback	CodeT5-small generates locally; a larger hosted model is the escalation path for low-confidence questions. Confidence is defined in §11.3.
Correcting data	An owner writes to the real table and the answer recalculates. A member writes to their own overlay, reads their own answers through it (§10.4), and the owner is asked to merge.
Safety layers	Four: intent gatekeeper, AST validator, database role separation, edit validator — plus the prompt rule in §11.4 that protects the explainer rather than the database.
Not the backend's job	Rendering, chart drawing, design tokens, browser session storage.

2. Roles

Two axes. The first has two values, decided by a question the person answers themselves.

2.1 Product role — who you are in your company

PRODUCT ROLE	WHO	SCOPE
owner	The first person to sign up from a domain nobody has registered, who says they are setting SpeakQL up. Creates the organisation.	Their own organisation: its databases, its people, its merge requests.
member	Everyone who signs up from that domain afterwards, and anyone redeeming an invitation.	Only the databases granted to them.

2.2 Database role — what you may do with one database

Stored on the access grant, not on the user, so the same person can be an analyst on one database and a viewer on another. Unchanged from Revision 3.

DATABASE ROLE	TABLES REACHABLE	SEES GENERATED SQL	MAY SUBMIT SQL	MAY CORRECT DATA
viewer	Only tables flagged `is_public`	No	No	No
analyst	Every table in that database	Yes	Yes — parsed by the same validator, no separate path	Yes

An owner implicitly holds analyst on every database in their organisation. This is a single check in `deps.require_db_access(connection_id, minimum_role)`.

2.3 Capability matrix

CAPABILITY	OWNER	MEMBER
Register a database connection	Yes	No
Upload a file, and choose where it lands	Yes	No
Invite people, approve a pending signup	Yes	No
Grant or revoke access, and set its database role	Yes	No
Mark a table read-only	Yes	No
Ask questions	Every database in the organisation	Granted databases only
Correct a value	Writes the real table immediately	Writes their own overlay; owner must merge
Approve or reject a member's correction	Yes	No
See what other people asked	Own organisation only	Own activity only
See another organisation	No — nobody can	No

2.4 Why there is no operator role

Revision 3 had a seeded account that could see every organisation, on the argument that a self-service path to the most powerful role is a vulnerability. Revision 4 resolved that differently and more simply: the most powerful role is scoped to one organisation, so reaching it by signing up grants nothing beyond the company you already belong to. The dangerous thing was never self-service — it was cross-tenant reach, and that no longer exists anywhere in the product.

> **The cost, stated plainly for the viva.**
> There is no support view. Diagnosing a customer problem means database access, not an application screen. If that is ever needed, it must be a seeded account that signup cannot reach — not a role anyone can select.

3. What the backend is responsible for

RESPONSIBILITY	MEANING
Identity	Issue and verify one-time codes; resolve an email domain to an organisation; decide from that domain whether the owner-or-member question is asked at all.
Access control	Enforce product role, organisation and database role on every call, and resolve every object identifier to its owner before using it (§6.1).
Provisioning	Register a database, invite a person, grant or revoke access at a stated database role.

RESPONSIBILITY	MEANING
Ingestion	Turn an uploaded file into a queryable, typed, introspected, indexed table, in the database the uploader named.
Understanding	Resolve follow-ups against thread history; ask back once when genuinely ambiguous.
Listening	Transcribe a spoken question and hold it for confirmation before it becomes a question.
Retrieval	Embed, search the workspace index, filter by grant and database role.
Generation	Produce candidate SQL from question, retrieved schema and persona, with a confidence score.
Validation	Reject anything that is not a single read-only SELECT; apply a stricter rule set to writes.
Routing	Accept the local answer or escalate to the fallback model.
Execution	Read under a read-only role, resolving the caller's own overlay if they have one; write under a separate, narrower one. Never both on one connection.
Correction	Apply an owner's edit to the real table; hold a member's edit in their overlay and raise a merge request; refuse a merge whose row moved underneath it.
Presentation data	Chart specification and explanation, at persona depth.
Learning	Record ratings and export negative ones as labelled training candidates.
Sharing	Stream a result as CSV, or a full report as PDF, under the same grant that produced it.
Oversight	Merge requests, access overview, and digest notifications with reminders.
Measurement	One `query_log` row per request, one `edit_log` row per accepted change, one `audit_log` row per administrative action.

4. Runtime topology

Four services. One command starts everything.

SERVICE	IMAGE BASIS	PORT	PURPOSE	PROFILE
db	`postgres:15`	5432	Three logical databases (§5)	always
api	`python:3.11-slim`	8000	FastAPI; CodeT5 and MiniLM in process; Whisper lazy; APScheduler for merge reminders	always
llm	`ollama/ollama`	11434	Gemma — the fallback generator and the explainer (§11.5)	local-llm
web	`node:20-alpine`	5173	React frontend	always

`llm` sits behind a Compose profile so one file serves both hosting choices: `make up` brings it up, `make up-hosted` omits it and `llm_client.py` calls a hosted endpoint instead. The client interface is identical either way, which is the whole reason that module exists. Adminer is gone from the submitted invocation.

5. Databases and schemas

DATABASE	SCHEMAS	CONTENTS	RUNTIME ACCESS
`speakql_meta`	`public`	Organisations, people, invitations, access grants, connections, schema registry, threads, messages, member edits, merge requests, and the three logs	API as owner — this is the application's own data
`<customer>_dw`	`public`, `member_edits`	The real warehouse, and one overlay schema per member holding corrections that are not yet merged, plus the overlay views in §10.4	`speakql_ro` reads `public` and the overlay views; `speakql_write` writes `public` under §10.2; `speakql_edits_rw` owns the overlay tables
`speakql_uploads`	`org_<id>`	Tables created from uploaded files that started a new database	`speakql_upload_ddl` to create and load; `speakql_ro` to query

The staging mirror is gone. An owner's edit goes to production deliberately, and a member's edit goes to a small overlay that holds only their pending rows — not a copy of the warehouse. A nightly resync, shadow hashes, drift fingerprints and conflict reconciliation all disappear with it.

5.1 Database roles

ROLE	GRANTS	USED BY	LOADED BY THE API
`speakql_owner`	Full rights everywhere	`bootstrap.sh`, Alembic	No — a separate DSN used only by the bootstrap script
`speakql_ro`	CONNECT, USAGE, SELECT on warehouse and upload schemas; SELECT on overlay **views** only	Every read	Yes
`speakql_write`	UPDATE and INSERT, only on tables marked editable, only when the statement is scoped by primary key	The edit path, and nothing else	Yes
`speakql_edits_rw`	All rights on its own overlay tables. No grant of any kind on `public`	Pending member corrections	Yes
`speakql_upload_ddl`	CREATE and INSERT within its own organisation's upload schema	File ingestion only	Yes

> **The sentence to say in the viva.**
>
> The read path holds no write privilege of any kind, and every write in the system goes through one path that is validated, scoped to a single primary key, restricted to tables the owner marked editable, and written to the edit log. Revision 3 could say "the application never writes to production"; once editing exists that sentence is no longer true, and saying it anyway would be the kind of claim a panel takes apart.

On the new grant in Revision 5. `speakql_ro` gaining SELECT on the overlay views does not weaken the sentence above. It is a read privilege on a read-only object, granted in the opposite direction to the assertion that protects the warehouse — which says `speakql_edits_rw` holds nothing on `public`, and is unchanged. §17 asserts both.

5.2 Metadata tables

TABLE	PURPOSE
`organisations`	One per registered company domain, created by the first owner — or one per person for a personal workspace. `kind` is `company` or `personal`; `domain` is **NULL** for personal, which is what keeps §7.2 true
`people`	Email, product role, organisation, verification state
`invitations`	Invite token, target organisation, database grants, expiry, single-use status (§7.5)
`access_grants`	person × connection, with `db_role` (analyst / viewer), `granted_by`, `revoked_at`
`connections`	A registered database or an uploaded dataset's home. Credentials encrypted (§9.3)
`schema_registry`	One row per column, with its description and whether it is public
`editable_tables`	Which tables may be corrected. Default: every table with a primary key
`member_edits`	A member's pending corrections — row key, column, before, after, note
`merge_requests`	One per pending correction, with its state (`open`, `merged`, `rejected`, `stale`) and who must decide
`uploaded_datasets`	One row per file that became a table, and where it landed
`clarifications`	One row per time the pipeline asked back
`feedback`	One row per thumbs up or down
`otp_attempts`	Failed-code counter and lockout window per account
`query_log`, `edit_log`, `audit_log`	The three logs, each with exactly one writer

Gone since Revision 3: `glossary_terms`, `staging_sync_state`, `staging_conflicts`.

6. Privileged path register

Every code path in the running application that holds more than SELECT, and what constrains it. A path that is added and does not appear here is a review failure, not an implementation detail.

PATH	ROLE USED	REACH	GUARD
Owner correction	`speakql_write`	One row of one table in one connection	`edit_validator.py`: table on the editable list, statement scoped by primary key, owner or analyst grant, one row affected or the transaction rolls back
Member correction	`speakql_edits_rw`	That member's overlay tables only	Same validator. Cannot reach `public` under any statement
Merge on approval	`speakql_write`	The one row the merge request names	Owner-authorised only; the request carries the row key it was raised against and cannot be redirected at another row; **and the live value must still equal the before value** (§10.5)
File ingestion	`speakql_upload_ddl`	The named target database, or a new one	Owner role; extension and content sniffing; row and size caps
Metadata writes	API owner on `speakql_meta`	The application's own data	Ordinary route authorisation
Bootstrap and migrations	`speakql_owner`	Everything	Offline. Its DSN is not in the API environment

Three module-level rules enforce this structurally rather than by convention:

- `db/tenant_engine.py`, `db/write_engine.py`, `db/edits_engine.py` and `db/uploads_engine.py` are separate modules with separate connection caches. No module imports another's cache.
- `executor.py` receives an engine, never a DSN, and cannot construct one.
- The bootstrap DSN is read from a variable `config.py` does not define, so an accidental import fails at startup rather than at runtime.

6.1 Object-level authorisation

New in Revision 5. A grant check asks *may this person use this database*. An object check asks *is this thing theirs to name at all*. They fail differently, so they are enforced separately. Guessing a `message_id` is the cheapest attack against this system and the one a panel is most likely to try.

IDENTIFIER	RESOLVED AGAINST	REFUSED WHEN
`message_id`	`messages` → `threads.person_id`	The thread is not theirs. An owner may read their own organisation's threads; a member only their own
`thread_id`	`threads.person_id`, `threads.organisation_id`	Different person, or different organisation
`merge_id`	`merge_requests.organisation_id`	The resolver is not the owner of the organisation that raised it
`connection_id`	`access_grants`	No grant, or `revoked_at` is set
`dataset_id`	`uploaded_datasets.organisation_id`	Different organisation
`clarification_id`	The message it belongs to	The parent message fails its own check

- **An object that is not yours and an object that does not exist return the same 403**, so existence is not discoverable by iterating identifiers. 404 is never used for an authorisation outcome.
- **Export inherits, it does not replay.** `GET /api/ask/{message_id}/export` resolves the identifier as above and then re-runs the grant check that produced the answer. An export is a second read, so access revoked between the answer and the download refuses the download.
- Every refusal at this layer writes one `audit_log` row with the identifier attempted. A run of them is the signature worth alerting on.

7. Authentication

Passwordless for both roles. Two code purposes with independent lifetimes.

PURPOSE	LENGTH	TTL	ATTEMPTS	TRIGGERED BY
`SIGNUP_VERIFY`	6 digits	10 minutes	5	`POST /api/auth/signup`
`LOGIN`	6 digits	5 minutes; issuing a new code invalidates any unused prior one	5	`POST /api/auth/login/request-otp`

7.1 Signup, and the question asked once

`POST /api/auth/signup {email, invite_token?}` resolves the address and returns one of **four** shapes. Revision 4 had three; the free-mail shape is new, and §7.2 is the rule that makes it safe.

SITUATION	RESPONSE	WHAT THE INTERFACE DOES
Invitation token present	`{first_user: false, organisation, grants}`	Asks nothing. The token fixes the organisation, the role and the database grants, and the account is created active. This shape is checked **first**, and works for any address including free-mail.
Free-mail address, no token NEW	`{first_user: true, personal: true}`	Asks nothing about companies. Creates a **personal workspace** — an organisation of one, with no domain.
Company domain nobody has registered	`{first_user: true, domain}`	Asks the one question: are you setting SpeakQL up for your company, or joining a team that is already here? The answer is sent with the verification.
Company domain already registered	`{first_user: false, organisation}`	Asks nothing. Says which company they are joining, and that they will join as a member.

Answering "I was invited" on a company domain nobody has registered is a dead end by construction — there is no organisation to join, and an invitation cannot conjure one, because an invitation is issued by an organisation.

An owner is therefore created exactly once per company domain, at the moment that domain is first seen, by somebody who said so. There is no path that produces a second owner for a company by accident.

7.2 Free-mail addresses, and the rule that keeps them safe

New in Revision 5. Revision 4 refused `gmail.com` outright, on the correct observation that it is not a company. But that refusal also turned away the individual analyst, the consultant and the student — and the thing that was actually dangerous was never the address. It was the lookup.

> **The one rule.**
>
> A free-mail address never resolves to an organisation by domain. If it did, `resolve_domain('gmail.com')` would return one organisation and every Gmail user in the world would land inside it, reading each other's warehouses. Keeping the domain lookup company-only is a one-line rule that removes that entirely — and costs the product nothing, because a personal workspace does not need a domain.

So a free-mail signup creates a **personal workspace** instead:

- An `organisations` row with `kind = 'personal'` and `domain` **NULL**, owned by exactly that person. Because the domain is null, no domain resolution can ever match it — the query is `WHERE domain = ? AND kind = 'company'`, and a null never equals anything.
- **Two people on `gmail.com` get two separate workspaces**, and neither can see the other. This is the isolation claim restated for the new shape, and §21 tests it directly.
- The only way a second person enters a personal workspace is an **invitation issued from inside it** — explicit, single-use, and naming the organisation. There is no implicit route in.
- A personal workspace is otherwise an ordinary organisation: the same two product roles, the same database roles on grants, the same validator, the same privileged path register. Nothing downstream of signup knows or cares which kind it is.
- An invitation from a **company** organisation may be sent to a free-mail address, and is honoured — the token fixes the organisation, so no domain lookup happens at all. A contractor on Gmail is exactly the case Revision 4 could not serve.

ORGANISATION KIND	DOMAIN	HOW SOMEONE JOINS	HOW IT IS CREATED
`company`	The registered domain	Domain match, or an invitation	The first person from that domain who says they are setting it up
`personal` NEW	`NULL`	Invitation only — never by domain	Any free-mail signup, automatically, with no question asked

`FREE_EMAIL_MODE` selects the behaviour: `personal_workspace` (the default), `invite_only` (a free-mail address may redeem an invitation but not create anything), or `blocked` (the Revision 4 behaviour, kept so an installation that wants it can have it). The variable replaces `BLOCK_FREE_EMAIL_DOMAINS`, which could only express two of those three.

7.3 Abuse controls on the login flow

CONTROL	VALUE	REASON
Attempts per code	5	Bounds a single guess run
Consecutive failed codes before lockout	5, then a 15-minute account lockout	Closes the request-a-new-code reset loop
Resend cooldown	60 seconds per email	Prevents inbox flooding
Request rate limit	10 per email per hour, 30 per IP per hour	Prevents using the login flow to mail-bomb a stranger

Codes are stored hashed, compared in constant time, and deleted on use. Six digits rather than four because four is ten thousand possibilities and the difference in user effort is one keystroke.

7.4 Rate limits beyond authentication

New in Revision 5. §7.3 bounds the login flow; these bound the paths that cost model time, disk or money. `ratelimit.py` is the single module, and every limit is a configuration value rather than a literal.

PATH	LIMIT	WHY
`POST /api/ask`, `/api/ask/voice`	60 per person per hour, 600 per organisation per hour	A question runs the local model and may escalate to a billed hosted one. This is the only path with a per-organisation ceiling as well as a per-person one
`POST /api/datasets`	20 per organisation per hour	Ingestion holds a DDL role, writes to disk and rebuilds an index
`GET /api/ask/{id}/export`	30 per person per hour	An export can be 50,000 rows
`POST /api/rows/correct`	120 per person per hour	Bounds a runaway client, not a person
`POST /api/merges/{id}/resolve`	Not limited	An owner action against a queue that is already bounded

A limit that fires returns **429** with the retry window, and still writes one `query_log` row with `route = none`. A rate-limited request that vanished from the log would be a hole in the measurement.

7.5 Invitations

An invitation token is single-use and expires after `INVITE_TTL_HOURS` (default 168). Redeeming it marks it `accepted` in the same transaction that creates the account, so two people cannot redeem one token by racing. A revoked or expired token returns the same refusal as a token that never existed.

8. Personas — depth of the answer

Persona decides what kind of answer you get and what an export contains — which is what people actually differ on.

Renamed in this revision. The two personas were `analyst` and `business`, displayed as "Analyst" and "Simple". `analyst` is already the name of a *database role* on a grant (§6.4), so the same word carried two unrelated meanings one line apart in this document. They are now **Workbench** and **Readout**, and the API field takes `workbench` or `readout`. The database roles are unchanged.

	READOUT	WORKBENCH
API field	`readout`	`workbench`
On screen	The number, the explanation, and the complete table of rows it came from	The number, a chart, the explanation, the SQL, and the complete table of rows
The data	Every row. Never truncated by persona	Every row
Chart	No	Yes
Generated SQL	Stripped from the response body server-side	Included
Explanation	One plain sentence naming the source columns	Short, assumes fluency
Export	CSV of every row behind the answer	CSV of every row, and a report carrying the charts and the written analysis

Persona never truncates data. It removes the chart, the SQL and the confidence reading; the row set behind the answer is identical in both. A persona that returned fewer rows would let two people quoting the same answer disagree about the data underneath it, and no amount of interface care recovers from that.

Persona is a display and input concern only. It never changes what may run: the validator, the executor and every access check behave identically for both. `readout` additionally strips the SQL from the response body rather than relying on the frontend to hide it, which is defence in depth and costs one line.

A person's persona defaults from their database role — an analyst grant defaults to Workbench, a viewer grant to Readout and cannot be changed, because a viewer may not see SQL at all.

9. Databases and uploads

9.1 Owner workflow

Register a connection or upload a file, invite colleagues, approve pending signups, grant access to specific databases at a stated database role, then keep an eye on merge requests.

9.2 File upload, and where it lands

The destination is asked, never inferred. A file is often a new subject area, and is often one more table in a subject area that already exists; guessing produces a sprawl of single-table databases nobody can join across.

STAGE	BEHAVIOUR
Accept	CSV, TSV, TXT, XLSX. Size capped by `MAX_UPLOAD_SIZE_MB` (default 25), rows by `UPLOAD_ROW_CAP` (default 500,000).
Sniff	Delimiter and structure detected from content, never from the extension. A low-confidence sniff is rejected with a clear message rather than guessed.
Ask where it goes	A new database — the uploader names it, and the file becomes its only table. Or a table in an existing database — the uploader picks the database and names the table, and it sits beside the tables already there so questions can span both. The interface shows the resulting name before anything is created.
Read	Excel read with `data_only=True`; formulas and macros are never evaluated. An uploaded file is untrusted input.
Name columns	Header row sanitised: lower-cased, non-alphanumerics to underscore, truncated, duplicates suffixed, reserved words quoted. The original header is preserved in `schema_registry.description` so retrieval still sees the human wording.
Type	Inferred from a 1,000-row sample: integer, numeric, date, timestamp, boolean, else text. Any column that falls back to text is reported to the uploader, not silently downgraded.
Load	COPY-based bulk load, never row-by-row inserts.
Register	A row in `uploaded_datasets`, a connection row if a new database was created, and introspection into `schema_registry`.
Reindex	**New in Revision 5.** Embeddings are computed for the new columns and the organisation's FAISS index is rebuilt, synchronously, before the upload is reported complete. Until Revision 5 this waited for somebody to run `make reindex`, which meant a freshly uploaded table could not be retrieved — and step 2 of the demonstration script in §22 depends on it being retrievable four steps later.
Re-upload	Replace-only in this version. Append is a v2 feature and the endpoint rejects it explicitly rather than silently choosing.

9.3 Registering an external connection

New in Revision 5. An owner supplies a host, port, database name and credentials. This is the one place the application is told to open a socket somewhere it did not choose, and it is checked accordingly.

STAGE	BEHAVIOUR
Resolve first	The hostname is resolved, and the **resolved address** is what gets checked — not the string. Checking the string lets a name that resolves to a private address straight through.
Refuse private ranges	Loopback, link-local, and RFC 1918 addresses are refused unless `ALLOW_PRIVATE_DB_HOSTS` is explicitly on, which it is only for a local demonstration against the seeded warehouse. The default is off.
Pin the address	The connection is opened to the address that was checked, not by re-resolving the name, so a name that answers differently a second time cannot redirect it.
Require TLS	`sslmode=require` at minimum. A connection that cannot negotiate TLS is refused rather than quietly downgraded.
Test	One `SELECT 1` under the supplied credentials with a 5-second timeout, before anything is stored. A connection that fails here is reported with the driver's message, which is almost always the fastest way to tell a wrong password from a wrong host.
Check privilege	The supplied account must hold no INSERT, UPDATE, DELETE or DDL privilege on the target schema. If it does, registration is refused and the offending tables are named.
Store	Credentials encrypted with `ENCRYPTION_KEY`. Plaintext exists only in the request body and the connection pool — never in a log, and never in any response, including the one that lists connections.

STAGE	BEHAVIOUR
Introspect and index	Schema into `schema_registry`, embeddings into the index, exactly as ingestion does.

> **Why the privilege check is not optional.**
> Every safety claim in §5.1 rests on the read path holding no write privilege. For an uploaded dataset that is true by construction, because SpeakQL created the role. For an external database it is only as true as the account the owner pasted in — so it is verified at registration, and re-verified by the assertions in §17. A customer who hands SpeakQL a superuser DSN would otherwise silently void the guarantee the whole design is built on.

10. Correcting data

The old design asked people to understand a mirror, a `source` parameter and a sync; this one asks them to notice a gap and type into it.

10.1 Where a gap is noticed

Incompleteness is a property of the data, so it shows up in the answer. A row with no value for a summed column is excluded from the total rather than counted as zero, the answer says how many rows that was, and the table underneath marks each missing cell. For anyone holding an analyst grant, that cell is also the way to fill it. There is no separate mode to enter and no toggle to remember.

10.2 An owner's correction

STEP	WHAT HAPPENS
1	The edit validator checks: the table is on the editable list, the statement is scoped by primary key, and exactly one row will be affected.
2	The write runs under `speakql_write` inside a transaction. If the row count is not exactly one, it rolls back.
3	One `edit_log` row records the before value, the after value, the person and the note.
4	The response carries the corrected row, and the open answer recalculates from it. The number on screen moves while the person is still looking at it.

There is no approval step. An owner approving their own change is theatre, and a step that exists only to be clicked through trains people to click through steps.

10.3 A member's correction

STEP	WHAT HAPPENS
1	The same validator runs, with the same three checks.
2	The row is written to that member's overlay under `speakql_edits_rw`, which holds no grant of any kind on `public`. The real table is untouched.
3	A `merge_requests` row is raised, carrying the row key, the before and after values and the member's note.
4	The member's own answers include their value from that moment, and say so — through the mechanism in §10.4. Everybody else continues to see the real data.
5	The owner is emailed. `POST /api/merges/{id}/resolve` decides it, under the staleness rule in §10.5.

Why a member's pending value appears in their own answers and nobody else's: they are the person who knows the real figure, and making them wait on an approval to use knowledge they already have is what drives people back to spreadsheets. Everyone else sees verified data, which is what protects the numbers other people quote.

10.4 How a pending value is read back

New in Revision 5, and the gap this revision exists to close. Revision 4 promised the behaviour in §10.3 step 4 and never said how a read achieves it. `speakql_ro` holds no grant on the overlay tables, and the answer is an aggregate, so there is nothing to patch after the fact.

One view per member per corrected table, created inside that member's overlay schema:

```
CREATE VIEW member_edits.v_<person>_<table> AS
SELECT p.<pk>,
       COALESCE(e.<col>, p.<col>) AS <col>, -- one per editable column
       ...
FROM public.<table> p
LEFT JOIN member_edits.e_<person>_<table> e USING (<pk>);
```

- `speakql_ro` is granted **SELECT on the views only** — never on the overlay tables underneath them, and never anything but SELECT. The §17 assertion that `speakql_edits_rw` holds no grant on `public` is untouched; this grant runs the other way, and it is read-only.
- The view is created the first time that member corrects a row in that table, and dropped when their last pending row for it is merged or rejected. A member with nothing pending has no views and reads `public` directly, like everybody else.
- **Resolution happens at execution, not generation.** `sql_generator.py` always produces SQL against `public`. `executor.py` rewrites the schema qualifier to the caller's overlay for exactly those tables where that caller has pending rows, and only on that caller's own request.
- An owner's request never resolves an overlay. Owners read `public`, always — which is what makes "as the owner, ask the same question, the number is unchanged" true in §22.
- `used_pending` on the `query_log` row is set when the rewrite touched at least one table. That is what makes the accuracy filter in §20 exact rather than approximate.

> **Why a view, and not a merge in the application.**
> Because the answer is an aggregate. Patching a SUM after the fact means reimplementing every aggregate the generator can emit — in Python, correctly, for every query shape — and getting it silently wrong. Letting the database compute over corrected rows means the arithmetic is the same arithmetic in both cases, and the only thing that differs is what the rows contain.

> **Why the rewrite lives in the executor, and not the generator.**
> Two reasons. The generator is a trained model and everything added to its input costs accuracy — pending data must never reach the prompt. And confining the rewrite to one function in one module means the read path can be tested exhaustively in one place, which is the same argument that put every safety rule in `validator.py`.

The cost, stated. A member accumulates one view per corrected table. `OVERLAY_MAX_TABLES_PER_MEMBER` caps this at 25; beyond it, new corrections are refused with a message asking them to get their queue merged. Somebody with pending corrections across twenty-five tables has a data problem their owner needs to see, and silently growing the schema would hide it.

10.5 The merge, and the staleness check

New in Revision 5. This one was a bug, not an omission. Revision 4 applied an approved merge to "the row the request names". Consider: a member proposes 42 for a row. The owner then corrects that same row directly to 50. The owner approves the merge — and 42 overwrites 50. A verified value is destroyed by an older proposal, and both writes look entirely legitimate in the edit log.

Every merge request already carries the before value it was raised against. At resolution:

STEP	WHAT HAPPENS
1	The live row is read under `speakql_write` **inside the same transaction** that would apply the change, so nothing can move between the check and the write.
2	Its current value is compared with the `before` value recorded on the request.
3	Equal → the merge applies, writes its `edit_log` row, and the member is told. This is the Revision 4 path, unchanged.
4	Different → **nothing is written.** The request moves to state `stale` and returns 409. The owner is shown three values side by side: what it was, what the member proposed, and what it is now.
5	From there the owner may reject it, or apply the member's value anyway — which raises a fresh request against the current `before`, so the decision is recorded against what was actually on the row.

The same rule covers two members proposing different values for one row. The first approval succeeds; the second finds the row changed and goes stale, so the second member is told what happened rather than the first member's approved value disappearing without either of them knowing.

> **Stale is not a failure state.**
> It is the system noticing that two people disagreed about one number — which is exactly the thing worth surfacing to an owner, and exactly the thing a silent last-write-wins would have buried.

10.6 Notifications and reminders

Every raised merge request notifies the organisation's owner through `app/mailer.py`, the single module that composes and sends mail. `NOTIFY_DIGEST_MINUTES` defaults to 10, so a fifty-row back-fill produces one email rather than fifty. A scheduled job then sends one reminder each morning while anything is still open — the failure this feature actually has is not a wrong merge but a forgotten one.

10.7 What was removed, and why it is worth saying out loud

REMOVED	WHY
The staging mirror as a query source, and the `source` parameter	It asked every person to hold two versions of the truth in their head and choose between them on every question. Almost nobody would, and the default meant most people never saw their own team's corrections at all.
Nightly resync, shadow hashes, drift fingerprints, conflict reconciliation	All of it existed to keep a copy of the warehouse honest. With no copy, none of it is needed. This is the single largest reduction in moving parts in the project.
The business glossary — authoring, validation, injection, embedding	It was a second place for the meaning of a word to live, needing its own screen, its own validation and its own failure mode. The explanation already names the columns a number came from, which lets the reader check the definition where it matters — on the answer.

11. Understanding the question

Two modules run before generation. Neither can widen what the access grant already allows — they only narrow or pause.

11.1 Follow-up memory

`core/context_resolver.py` rewrites "now break that down by region" into a self-contained question using the previous turn's metric, tables and filters. Both versions are stored in `messages` — what the person typed and what the system understood it to mean — so a wrong answer can be diagnosed without guesswork.

11.2 Clarification

When retrieval surfaces two similarly scored candidate metrics, or generation confidence lands in the uncertain band, the pipeline stops and emits `clarification_needed` with two to four options. The choice comes back referencing a `clarification_id`, generation re-runs with it folded into context, and the exchange is logged.

One round, then commit. `CLARIFY_MAX_ROUNDS` is 1. A question still ambiguous after one round gets the best-guess answer with its assumption stated in the explanation. One clarification is good service; three is a broken interface.

11.3 Confidence — what the number actually is

New in Revision 5. Six decisions read this number and no backend section defined it. The definition lives in Complete Logic §2 and §5; it is restated here because routing, clarification and the seal band all depend on it, and none of them should have to look it up in another document.

Beam search returns the best candidate and its cumulative log-probability. **Confidence is that score divided by the number of generated tokens, mapped to 0–1.** Length normalisation is not cosmetic: raw sequence probability falls monotonically as output grows, so without it the router systematically prefers short queries — which on this task means queries that are missing a join.

READ BY	WHAT IT DOES WITH IT
`router.py` (§14 step 8)	Below `CONFIDENCE_THRESHOLD`, escalate to Gemma (§11.5) and validate again
`clarifier.py` (§11.2)	Inside an uncertain band around the threshold, ask back instead of answering
The seal band	Displayed to the Workbench mode beside route, latency and row count
`query_log`	Retained per request, so the threshold sweep can be run over real traffic

`CONFIDENCE_THRESHOLD` is 0.55 until the week-9 sweep replaces it. The sweep plots share-served-locally against execution accuracy over the development set; the knee of that curve is the chosen value, and the plot goes in the results chapter.

> **What confidence is not.**
> It is not a probability that the SQL is correct. It is the model's confidence in its own decoding, which correlates with correctness but does not measure it. Say exactly this if asked — claiming calibration you have not measured is a claim a panel can ask you to prove, and you would not be able to.

11.4 Result values are data, never instructions

New in Revision 5. The four layers in §6 protect the database. This protects the explainer, and nothing in Revision 4 covered it.

`explainer.py` and `visualiser.py` receive rows read out of a customer's warehouse. A cell containing *"ignore your instructions and ..."* is untrusted input arriving at a language model. Three rules:

- Result rows are passed in a separate, clearly delimited data section of the prompt. They are never concatenated into the instruction section.
- The explainer's output is **text only**. It cannot cause a query, a write, an email or a tool call, so the worst outcome is a misleading sentence rather than an action. This is the structural half of the defence and it is worth stating first.
- The explanation names the columns a number came from, and those names come from `schema_registry` — not from the model's output. The checkable part of an explanation therefore cannot be forged by data.

The adversarial set in §21 includes cell values that attempt this. Zero must change the explainer's behaviour.

11.5 The fallback model, named

New in this revision. Every earlier revision called this "the fallback model" or "a larger hosted model" and never said which one — which left the single largest component in the system written down as a capability rather than a decision. A panel can ask "which model?", and "a larger one" is not an answer.

It is **Gemma**, Google's open-weight model, served by the `llm` container through Ollama. `LLM_MODEL` is `gemma3:4b` quantised, roughly 3 GB resident; `gemma2:2b` is the drop-down if the demonstration machine is short of memory. No other module names a model — `llm_client.py` reads `LLM_MODE` and `LLM_MODEL`, and presents one interface over both the local container and a hosted endpoint, which is the entire reason that module exists.

GEMMA IS ASKED TO	STEP	WHAT HAPPENS TO WHAT IT PRODUCES
Generate SQL, when CodeT5 lands below `CONFIDENCE_THRESHOLD`	§14 step 8	**The same sqlglot validator runs on it, unchanged.** There is no second, softer path for the fallback. A model with more parameters is given no more trust
Explain a result in prose	§14 step 11	Text only, under the three rules in §11.4. It cannot cause a query, a write, an email or a tool call, so its worst outcome is a misleading sentence rather than an action

> **What Gemma is never asked.**
>
> Any safety decision, anywhere. Whether a statement may run is decided by a deterministic AST parser checked against the schema registry, and that decision is byte-for-byte identical whether the SQL came from CodeT5, from Gemma, or from a person typing it by hand. A parser can be read, tested, and shown to a panel line by line; a model can only be sampled. **Gemma generates and explains. sqlglot decides.**

Where this stands for the second presentation. The routing design above is CodeT5 first and Gemma on escalation. Until CodeT5 is fine-tuned, that router has one arm: **Gemma is the generator for every question**, with retrieval pretrained and zero-shot, and the validator, the executor and the correction path all real. Say that plainly rather than letting it be discovered — it is the second row of the five-configuration study the synopsis already promises, demonstrated early, not a shortfall.

It is also what makes that study possible at all. Three of its rows — CodeT5 alone, Gemma alone, and the hybrid routed at 0.55 — exist only because the fallback is a named, runnable, locally hosted model instead of an unspecified escalation to something billed.

12. API contract

New in Revision 5. Revision 4 named endpoints in passing across five sections, and §14 was a file inventory rather than a contract. The Frontend Specification calls this the most expensive seam in the project — the mock and the real response must share one shape, and if they drift the interface gets rewritten. This is that shape.

Every route below requires a valid session except `signup`, `login/request-otp` and `health`. Every route taking an identifier resolves it under §6.1 first.

12.1 Authentication

METHOD AND PATH	REQUEST	RESPONSE
`POST /api/auth/signup`	`{email, invite_token?}`	`{first_user, personal?, domain?, organisation?, grants?}`
`POST /api/auth/verify`	`{email, code, role_choice?}`	`{access_token, refresh_token, person, organisation}`
`POST /api/auth/login/request-otp`	`{email}`	`{sent, ttl_seconds, resend_after}`
`POST /api/auth/login/verify`	`{email, code}`	`{access_token, refresh_token, person, organisation}`
`POST /api/auth/refresh`	`{refresh_token}`	`{access_token}`
`POST /api/auth/logout`	—	`204`
`GET /api/auth/me`	—	`{person, organisation, grants[], persona}`

12.2 Organisation — owner only

METHOD AND PATH	REQUEST	RESPONSE
`GET /api/org`	—	`{organisation, database_count, people_count, open_merges}`
`GET /api/org/people`	—	`[{person, product_role, state, grants[]}]`
`POST /api/org/invitations`	`{email, grants[]}`	`{invitation, expires_at}`
`POST /api/org/people/{id}/approve`	—	`{person}`
`POST /api/org/grants`	`{person_id, connection_id, db_role}`	`{grant}`
`DELETE /api/org/grants/{id}`	—	`204`
`GET /api/org/access`	—	`{people[], connections[], grid[][]}` — the two-axis overview

12.3 Databases, uploads and schema

METHOD AND PATH	REQUEST	RESPONSE
`GET /api/connections`	—	`[{id, name, origin, table_count, row_count, db_role}]`. Never carries credentials
`POST /api/connections`	`{name, host, port, database, user, password}`	`{connection}` after every check in §9.3
`POST /api/datasets/preview`	multipart file	`{columns[], inferred_types[], suggested_name, warnings[]}`
`POST /api/datasets`	multipart file + `{destination: {mode: new|existing, connection_id?, database_name?, table_name}}`	`{dataset, created_object, rows_loaded, type_report[], reindexed}`
`GET /api/schema/{connection_id}`	—	`{tables[{name, columns[], is_public, editable}]}` at the caller's database role

12.4 Asking

METHOD AND PATH	REQUEST	RESPONSE
`POST /api/ask`	`{question, connection_id, thread_id?, persona}`	SSE stream — §12.6
`POST /api/ask/voice`	multipart audio, capped by `MAX_AUDIO_MB`	`{transcript, confidence}`. Never auto-submitted
`POST /api/ask/{message_id}/clarify`	`{clarification_id, option_id}`	Resumes the SSE stream at step 6
`POST /api/ask/{message_id}/feedback`	`{rating, comment?}`	`204`
`GET /api/ask/{message_id}/export`	`?format=csv|report`	Streamed file. `report` is Workbench only
`GET /api/threads`	—	`[{id, title, updated_at}]` — own threads; an owner may pass `?person_id=`
`GET /api/threads/{id}`	—	`{thread, messages[]}`

12.5 Corrections and merges

METHOD AND PATH	REQUEST	RESPONSE
`POST /api/rows/correct`	`{connection_id, table, pk_column, pk_value, column, value, note}`	`{row, mode: applied|pending, merge_id?, recalculated}`
`GET /api/merges`	—	`[{id, table, row_key, before, proposed, live, note, member, state, raised_at}]`
`POST /api/merges/{id}/resolve`	`{decision: approve|reject|apply_anyway, reason?}`	`{state: merged|rejected|stale, live_value?}` — 409 with the live value when stale (§10.5)
`GET /api/health`	—	`{status, models_ready, fallback_reachable, version}`

12.6 The streaming envelope

`POST /api/ask` returns Server-Sent Events. One `stage` event per pipeline step gives the interface something honest to show while the model is running, and exactly one terminal event always arrives.

EVENT	PAYLOAD	WHEN
`stage`	`{step, name}`	Each of the twelve steps in §14, so progress is real rather than a spinner
`clarification`	`{clarification_id, options[]}`	Step 5 pauses. Terminal until answered
`blocked`	`{rule, layer, statement?}`	Step 7 rejects. `statement` is absent when layer 1 fired, because no SQL was generated
`refusal`	`{reason, would_need}`	Step 4 found nothing that could answer it
`answer`	`{headline, rows[], truncated, chart?, explanation, sql?, seal}`	Step 11
`error`	`{code, message, retryable}`	Any step, including a statement timeout
`done`	`{message_id, latency_ms}`	Always last, on every path including the failures above

The `seal` object is `{route, confidence, latency_ms, row_count, used_pending, truncated}`. It is the same object the frontend renders as the seal band, and every field in it is one the answer must be able to justify.

12.7 Errors

One envelope: `{error: {code, message, rule?, retry_after?}}`.

STATUS	MEANS	NOTES
`400`	Malformed request	
`401`	No session, or an expired one	The interface refreshes once, then returns to sign-in
`403`	No grant, wrong product role, or an object that is not yours	**Also returned for an object that does not exist**, so existence is not discoverable (§6.1)
`409`	A merge whose row moved	Carries the live value (§10.5)
`413`	Upload or audio over the cap	
`422`	The validator refused the generated SQL	Carries `rule` and `layer`. Not a 500 — the system worked correctly
`429`	Rate limited	Carries `retry_after` (§7.4)
`503`	Fallback model unreachable and no valid local answer	Stated to the person, never silently degraded

13. Voice, feedback and export

Voice. `POST /api/ask/voice` accepts audio under the caller's existing session and grants, capped by `MAX_AUDIO_MB` (default 10) and `MAX_AUDIO_SECONDS` (default 120). A local Whisper checkpoint transcribes on the fast path, escalating to a hosted provider on poor audio or low confidence. The transcript is shown back for a one-tap confirm or edit and is never auto-submitted — a misheard word silently reaching the generator is a worse failure than a one-second confirmation. Whisper lazy-loads on the first voice request and unloads after `STT_IDLE_UNLOAD_MIN` (default 15), so an installation that never uses voice never pays its memory cost.

Feedback. `POST /api/ask/{message_id}/feedback` records a rating and optional comment. `scripts/export_feedback_for_training.py` pulls negatives into a file `ml/` can train against — still files passed between the two sides, never an import. A rating tells you where to look, not what the right answer was, so the export feeds a human review step, not a retraining trigger.

Export. `GET /api/ask/{message_id}/export?format=csv|report` streams the result under the same grant that produced it, re-checked at download time (§6.1). `csv` is every row behind the answer and is available in both personas. `report` is Workbench only and embeds the charts and the written analysis alongside the data. Bounded by `EXPORT_MAX_ROWS` (default 50,000) and recorded as a flag on the existing `query_log` row rather than in a parallel table.

14. Request lifecycle

Twelve steps, unchanged in count since Revision 4. Step 9 gains the overlay resolution from §10.4.

#	MODULE	PRODUCES	ON FAILURE
1	`deps` + `rbac`	Person, organisation, grant, database role, persona — and every identifier in the request resolved under §6.1	401 invalid token; 403 no grant, a foreign object, or a viewer attempting a correction
2	`question_handler`	Orchestration; opens the stream	—
3	`context_resolver`	Self-contained question	No antecedent, passes through unchanged
4	`schema_retriever`	Tables and columns, filtered by grant and database role	Empty, unanswerable response
5	`clarifier`	Green light, or `clarification_needed`	Ambiguous, stream pauses and resumes at step 6 with the choice
6	`sql_generator`	Candidate SQL and confidence (§11.3)	Timeout, escalate
7	`validator` or `edit_validator`	Accept, or reject naming the rule	Reject, blocked response, nothing executed
8	`router`	`local` or `fallback`	Fallback down, serve local if valid, else state the failure
9	`executor`	Rows, from the engine the mode selected — **resolving the caller's own overlay views if they hold pending rows (§10.4)**	Timeout or cap, partial suppressed, limit reported
10	`visualiser`	Chart specification (Workbench) or row set (Readout)	Unknown shape, table only
11	`explainer`	Explanation at persona depth, over delimited data (§11.4)	Failure, answer without explanation, logged
12	`query_log`	Exactly one row	Must never be skipped

Six invariants.

- Organisation and connection identity come from the session, never the request body, and are re-asserted at step 9.
- Step 12 runs on every path — rejections, refusals, crashes, rate limits, and clarifications that were asked but never answered.
- Reads and writes against the same warehouse never share a connection pool or a database role.
- Steps 3 and 5 only narrow or pause; neither can widen the grant established at step 1.
- No read path holds any write privilege, and every write is scoped to one primary key on a table the owner marked editable.

- **Overlay resolution at step 9 applies only to the caller's own pending rows, and never to an owner.** Nothing about pending data reaches the generator at step 6.

15. File inventory

Changes since Revision 4 are marked.

AREA	FILES
speakql/	docker-compose.yml, .env.example, Makefile, README.md
backend/app/	main.py, config.py, deps.py, rbac.py, mailer.py, ratelimit.py CHANGED — now covers ask, upload and export (§7.4), object_access.py NEW — the §6.1 resolvers
auth/	routes.py signup, verify, login, refresh, logout, me · otp.py, jwt_handler.py, public_domains.py, domain_resolver.py, invitations.py CHANGED — single-use and expiry
api/	routes_ask.py, routes_schema.py, routes_threads.py, routes_user.py, routes_org.py, routes_datasets.py, routes_rows.py, routes_merges.py CHANGED — the staleness check, routes_connections.py NEW — external registration (§9.3), schemas.py CHANGED — now the single source of the §12 contract
core/	question_handler.py, context_resolver.py, clarifier.py, schema_retriever.py, sql_generator.py, validator.py, router.py, edit_validator.py, merge.py CHANGED — staleness, executor.py CHANGED — overlay resolution, file_ingest.py CHANGED — reindex, dataset_loader.py, speech_to_text.py, export.py, visualiser.py, explainer.py CHANGED — delimited data section, llm_client.py
db/	session.py, tenant_engine.py, entities.py, schema_index.py, introspect.py, write_engine.py, edits_engine.py CHANGED — owns overlay views, uploads_engine.py, host_guard.py NEW — §9.3 address checks
logging/	query_log.py, edit_log.py, audit_log.py, notifications.py
Other	data/, ml/, scripts/

REMOVED since Revision 3: core/glossary.py, api/routes_glossary.py, core/staging_sync.py, db/staging_engine.py, db/sync_engine.py.

15.1 Rules about this tree

- validator.py and edit_validator.py are the only places safety rules live. A check anywhere else is a bug.
- The three log writers are the only writers of their tables.
- The four engine modules never import each other's caches.
- schemas.py is the only definition of a request or response shape. If §12 and schemas.py disagree, schemas.py is wrong, because the document is what the frontend was built against.
- ml/ and app/ communicate through files only.

16. Configuration

GROUP	VARIABLES
Database	POSTGRES_USER, POSTGRES_PASSWORD, META_DB_URL, BOOTSTRAP_DSN (bootstrap only, absent from the API environment)
Auth	JWT_SECRET, JWT_ALG, ACCESS_TOKEN_MINUTES, REFRESH_TOKEN_DAYS
OTP	OTP_LENGTH=6, OTP_SIGNUP_TTL_SECONDS=600, OTP_LOGIN_TTL_SECONDS=300, OTP_MAX_ATTEMPTS=5, OTP_MAX_FAILED_CODES=5, OTP_LOCKOUT_MINUTES=15, OTP_RESEND_COOLDOWN_SECONDS=60
Signup	FREE_EMAIL_MODE=personal_workspace|invite_only|blocked CHANGED — replaces BLOCK_FREE_EMAIL_DOMAINS, which could only express two of the three, INVITE_TTL_HOURS=168
Rate limits NEW	RL_ASK_PER_PERSON_HOUR=60, RL_ASK_PER_ORG_HOUR=600, RL_UPLOAD_PER_ORG_HOUR=20, RL_EXPORT_PER_PERSON_HOUR=30, RL_CORRECT_PER_PERSON_HOUR=120
Connections NEW	ALLOW_PRIVATE_DB_HOSTS=false, DB_CONNECT_TIMEOUT_SECONDS=5, REQUIRE_DB_TLS=true
Encryption	ENCRYPTION_KEY
Models	MODELS_DIR, RETRIEVER_PATH, GENERATOR_PATH, CONFIDENCE_THRESHOLD=0.55, CONFIDENCE_UNCERTAIN_BAND=0.08 NEW
Fallback	LLM_MODE=local|hosted, LLM_ENDPOINT, LLM_MODEL
Editing	EDITABLE_DEFAULT=all_with_pk, EDIT_REQUIRE_NOTE=true, OVERLAY_MAX_TABLES_PER_MEMBER=25 NEW
Merges	MERGE_NOTIFY=true, NOTIFY_DIGEST_MINUTES=10, MERGE_REMINDER_HOUR=9
Uploads	MAX_UPLOAD_SIZE_MB=25, UPLOAD_ALLOWED_EXTENSIONS, UPLOAD_TYPE_INFERENCE_SAMPLE_ROWS=1000, UPLOAD_ROW_CAP=500000
Voice	STT_MODE=local|hosted, STT_IDLE_UNLOAD_MIN=15, MAX_AUDIO_MB=10 NEW, MAX_AUDIO_SECONDS=120 NEW
Understanding	CLARIFY_MAX_ROUNDS=1
Export	EXPORT_MAX_ROWS=50000
Limits	MAX_ROWS=5000, STATEMENT_TIMEOUT_MS=10000, RETRIEVAL_TOP_K=25
App	FRONTEND_ORIGIN, LOG_LEVEL

Removed since Revision 3: ORG_AUTO_APPROVE, STAGING_SYNC_INTERVAL_MIN, STAGING_EDITABLE_DEFAULT, ADMIN_EMAIL and ADMIN_BOOTSTRAP_SECRET. Removed in Revision 5: BLOCK_FREE_EMAIL_DOMAINS, superseded by FREE_EMAIL_MODE.

config.py validates at startup and exits if a required variable is missing. .env goes into .gitignore in the first commit, before any secret exists.

17. Startup sequence

STEP	ACTION	GUARD
1	db starts	Health check passes
2	Three databases created if absent; member_edits and org_<id> schemas created	psql connects to each
3	Alembic migrations against speakql_meta	Head reached
4	Warehouse DDL and seed applied if absent	Row counts non-zero

STEP	ACTION	GUARD
5	Five roles created and granted	Six assertions, listed below
6	Metadata seeded: two organisations, one owner and two members each, grants at both database roles, one pending invitation, two open merge requests, **one stale merge request**	Both roles sign in
7	Schemas introspected into `schema_registry`; editable tables computed from primary keys	Row per column
8	Embeddings computed, FAISS index written per organisation	Index files present
9	CodeT5 and MiniLM loaded and warmed; fallback reachability checked	`/api/health` reports models ready
10	Merge reminder scheduler starts	First run logs a timestamp
11	API accepts traffic	Health returns 200

The six privilege assertions, run in CI as well as at boot. Revision 4 had four; two are new.

- `speakql_ro` reads every warehouse schema and cannot write to any of them.
- `speakql_write` can update a row on an editable table and is refused on a table not on that list.
- `speakql_edits_rw` writes its overlay and holds no grant of any kind on `public`.
- The API environment contains no `BOOTSTRAP_DSN`, so an accidental import fails at startup rather than at runtime.
- **NEW —** `speakql_ro` **can SELECT an overlay view and is refused on the overlay table beneath it**, which is what keeps the §10.4 grant read-only in practice and not merely by intent.
- **NEW — every registered external connection's stored account still holds no write privilege**, re-checked at boot rather than trusted from registration time (§9.3).

18. How to run

```
cp .env.example .env # JWT_SECRET, ENCRYPTION_KEY
./scripts/fetch_models.sh # CodeT5, MiniLM, Whisper
docker compose --profile local-llm up --build
# hosted fallback instead: omit the profile and set LLM_MODE=hosted
docker compose up --build
```

MAKE TARGET	DOES
`make up`	Full stack with the local `llm` service
`make up-hosted`	Full stack, hosted fallback
`make dev`	Database in Docker, API on the host with reload
`make bootstrap`	Migrate, seed, introspect, index
`make reindex`	Rebuild vector indexes. No longer required after an upload (§9.2), and kept for schema changes made outside SpeakQL
`make test`	Full pytest suite
`make demo`	Reset, bootstrap, and run the demonstration script (§22)

19. Memory and performance

CONFIGURATION	RESIDENT MODELS	API PEAK	WHOLE STACK	SUITABLE FOR
A — hosted Gemma, hosted speech	MiniLM, CodeT5	~1.6 GB	~2.4 GB	8 GB laptop, network required
B — hosted Gemma, local Whisper *(recommended for the demo)*	MiniLM, CodeT5, Whisper on demand	~2.2 GB	~3.2 GB	8 GB laptop
C — local `gemma3:4b`, local Whisper	All of the above plus Gemma in the `llm` container	~2.2 GB api + ~2.8 GB llm	~6.5 GB	16 GB machine only
C′ — local `gemma2:2b`, local Whisper	As C, with the smaller Gemma	~2.2 GB api + ~1.8 GB llm	~5.4 GB	8 GB machine, at some accuracy cost

CONCERN	DECISION
Inference blocking the event loop	Every model call runs in a thread pool executor. The single most likely performance bug in this stack.
Cold start	CodeT5 and MiniLM warmed at boot; Whisper warmed on first use.
Long queries	10-second statement timeout.
Large results	5,000 rows on screen, 50,000 on export, both reported not silently applied.
Connection churn	One pooled engine per connection per purpose, cached.
A correction under load	Writes take a row lock for the duration of one statement. Scoped by primary key, so two people correcting different rows never contend.
Overlay resolution NEW	One LEFT JOIN per corrected table, on the primary key, against a table holding a member's pending rows — tens of rows, not thousands. The join is indexed and the cost is not measurable against a 10-second timeout.
Fallback down	Retry with backoff, then degrade to local-only.

20. Logging and measurement

Application logs go to stdout, structured, ephemeral. Three tables carry the record, each with exactly one writer: `query_log`, `edit_log`, `audit_log`, plus `feedback`.

COLUMN OF `QUERY_LOG`	PURPOSE
`question`, `resolved_question`	As typed, and as the context resolver understood it
`generated_sql`	Retained even when rejected
`route`	`local`, `fallback`, or `none`
`valid`, `reject_reason`	Validator outcome and which rule fired
`mode`	`read` or `correct`
`persona`	`workbench` or `readout`
`clarified`	Whether step 5 asked back

COLUMN OF QUERY_LOG	PURPOSE
used_pending	Whether the answer included the asker's own unmerged correction — set by the overlay rewrite in §10.4
exported	Whether the answer was downloaded, and in which form
confidence	The §11.3 number, retained so the threshold sweep can run over real traffic
latency_ms, prompt_tokens, output_tokens	Cost and performance
created_at	Ordering

The rule that protects the results chapter.

Every accuracy figure in the report is computed over `mode = 'read' AND used_pending = false`. An answer that included the asker's own unmerged correction is answering over data they typed, and counting it would overstate the model. Two further figures are reported separately and honestly: clarification rate, and the share of questions served locally.

21. Testing

CATEGORY	COVERS	TARGET
Unit	Validator rules, edit-validator rules, domain resolution, OTP lockout, type inference, chart selection, confidence normalisation	One passing and one failing case per rule
Signup	Unseen company domain asks the question; known domain does not; invitation fixes role and grants; an invitation cannot be redeemed twice; a second owner cannot be created for a domain	Every branch of §7.1 covered
Free-mail NEW	**Two signups from the same free-mail provider land in different organisations**, and neither can read the other; a free-mail address is never returned by domain resolution; an invitation carries a free-mail address into a company organisation; a personal workspace can invite; FREE_EMAIL_MODE=blocked restores the Revision 4 refusal	The isolation test in §21 runs against a personal workspace as well as a company one
RBAC matrix	Every cell of §2.3	A seeded person of each role receives the correct 200 or 403
Object access NEW	A guessed message_id, thread_id, merge_id and connection_id from another organisation; an export after the grant was revoked	403 in every case, indistinguishable from a non-existent id, and one audit_log row each
Isolation	Cross-organisation access	403 in every case, logged
Privilege	The six assertions in §17	All six pass in CI
Correction	Owner write lands and recalculates; member write lands only in the overlay; unscoped update refused; non-editable table refused; viewer refused	The real table is byte-identical after every refused case
Overlay NEW	A member's answer resolves their pending value; the owner's identical question does not; the view is dropped when the queue empties; used_pending is true exactly when the rewrite fired; the cap refuses the twenty-sixth table	A checksum proves public never changed, and the aggregate is correct in both directions

CATEGORY	COVERS	TARGET
Merge	Raise, approve, reject; approved value appears for everyone; rejected value disappears for the member; reminder fires while open	Both outcomes and the reminder
Staleness NEW	Owner edits a row, then approves an older merge against it; two members propose different values and both are approved in turn	409 and state `stale` in both cases, and the newer value survives
Connections NEW	A host resolving to a private address; a host that refuses TLS; an account holding UPDATE	Refused at registration, with the reason named
Rate limits NEW	The ask, upload and export ceilings	429 with `retry_after`, and a `query_log` row still written
Ingestion	Malformed CSV, wrong extension, formula-bearing XLSX, duplicate headers, 100k-row load, both destinations, and retrievability immediately after load	Clean rejection or correct load, never a partial table
Adversarial	Report Table 13.2, write-path attacks, **and cell values attempting prompt injection (§11.4)**	Zero unsafe executions, zero changes to explainer behaviour
Model	Retrieval recall, execution accuracy	Held-out set only
Acceptance	The demonstration script (§22)	Passes from a fresh reset

22. Demonstration script

`make demo` runs this end to end. It is also the acceptance test for the whole project, and it is the same sequence as §18 of the Frontend Specification — twenty-two steps in both documents.

- Sign up with an unseen domain. The one question is asked; answer "I am setting this up" and become the owner.
- Register an external connection. The host check, the TLS requirement and the privilege check all pass, and are shown passing.
- Upload a CSV and send it to a new database.
- Upload a second file and send it to a table inside an existing database. Show the resulting name before it is created.
- Ask a question about the table just uploaded. **It is retrievable immediately** — no `make reindex` (§9.2).
- Invite an employee; approve a pending signup; grant one database as analyst and one as viewer.
- Sign up a second person from the same domain. No question is asked; they become a member.
- As the member, ask a confident question. Workbench mode shows the chart, the explanation and the SQL, and the seal band carries the confidence from §11.3.
- Switch to Simple. The SQL is gone from the response body, and the answer is the number and its rows.
- Export from Readout: every row as CSV. Export from Workbench: the report, with charts and analysis.
- Ask an ambiguous question. One clarification round, then a correct answer.
- Ask a follow-up; it is resolved against the thread.
- Ask a destructive question. The validator rejects it, naming the rule; nothing reaches the database.
- Reach for another organisation. 403, and an audit row. Then paste another organisation's `message_id` into an export URL — the same 403, and another audit row.
- Ask a question whose data is incomplete. The answer says how many rows were excluded and marks them.
- As the member, fill the gap in the table. It lands in their overlay; their own answer recalculates; the owner is emailed.
- As the owner, ask the same question. The real number is unchanged, because nothing was merged.
- As the owner, approve the merge. Ask again: the number now includes it, for everyone.

- As the owner, fill a different gap. It writes the real table immediately and the answer recalculates with no approval step.
- **Show the staleness check.** Approve the seeded merge request whose row the owner has since edited. It refuses, shows all three values, and the newer value survives.
- Ask by voice; the transcript is shown, confirmed, answered.
- Rate an answer down.

23. Build order — three lanes, ten weeks

The critical path is unchanged: the machine-learning work. The product surface runs in parallel and never blocks it.

WEEK	LANE A — MODELS AND PIPELINE	LANE B — PRODUCT AND INTERFACE	LANE C — DATA, EVALUATION, TESTS
1	Spider preparation; retriever training	Repo skeleton, compose, health endpoint	Warehouse DDL, seed both organisations
2	Retriever recall table produced	Auth: two-purpose OTP, lockout, JWT, domain resolution	Five roles and the six privilege assertions
3	Schema introspection and FAISS index	The signup question, invitations, grants; `object_access.py`	Synthetic pair generation; isolation and object-access tests
4	Generator training, stage one	Organisation and people routes; §12 contract frozen	Pair verification; upload fixtures
5	Generator training, stage two	File ingestion with destination and reindex; external connection registration	Ingestion tests; RBAC matrix tests; host-guard tests
6	Validator, edit-validator, router; confidence normalisation	Corrections: owner path, member overlay and its views, merge requests	Correction, overlay and staleness tests; real-table checksum test
7	Executor including overlay resolution; `/api/ask` end to end via curl	Clarifier, context resolver	Adversarial set, both paths, plus prompt-injection cells
8	Streaming endpoint and the §12.6 envelope; fallback wiring	Voice, export in both forms, notifications, feedback	Evaluation harness; ablation runs begin
9	Threshold sweep; ablation completed	Frontend integration across both roles	Failure analysis; full test suite green
10	Model cards, reproduction notes	`make demo` green end to end	Results tables; report amendments (§25)

Two rules protect the schedule. Lane A never waits on Lane B — the pipeline is tested with curl, not through the interface. And any slip in Lane B is absorbed by simplifying a feature's surface, never by deleting it: console mail instead of SMTP, a seeded merge request instead of a raised one. The feature still demonstrates; only its polish moves.

24. Decisions

DECISION	DEFAULT CHOSEN
Who becomes the owner	The first person from an unseen domain who says so. Asked once, never again for that domain.
Free-mail addresses CHANGED	Accepted, and given a personal workspace with a null domain. Never matched by domain resolution, so two people on the same provider are two tenants. `FREE_EMAIL_MODE` can restore the refusal.
Owner corrections	Applied immediately, no approval, one edit-log row.

DECISION	DEFAULT CHOSEN
Member corrections	Overlay plus merge request. Visible in their own answers while pending, through the overlay views in §10.4.
How a pending value is read NEW	A `COALESCE` view per member per table, resolved in the executor. Not an application-side merge, because the answer is an aggregate.
A merge whose row moved NEW	Refused, marked `stale`, and shown to the owner with all three values. Never last-write-wins.
Merge reminders	A digest every ten minutes, then one reminder each morning while anything is open.
Editable tables	Every table with a primary key. The owner may mark any of them read-only.
Tables without a primary key	Never editable, by anyone. A change that cannot be pinned to one row cannot be checked.
Upload destination	Always asked. Never inferred.
Reindex after upload NEW	Synchronous, inside ingestion. A table you cannot ask about is not loaded.
Re-upload	Replace only; append rejected explicitly.
Persona default	From the database role. A viewer is Simple and cannot change it.
Private database hosts NEW	Refused unless `ALLOW_PRIVATE_DB_HOSTS` is set, which only the local demonstration does.
A foreign object identifier NEW	403, identical to one that does not exist. Never 404.
Fallback hosting	Local for development, hosted for the demonstration machine unless it has 16 GB.
`CONFIDENCE_THRESHOLD`	0.55 until the sweep produces the tuned value.
Clarification rounds	One.
Promote-to-production review queue for owners	Not built. Owners write directly; that is the point of the role.

Still genuinely open, none blocking: whether the demonstration machine has 16 GB, which selects configuration B or C; and whether the hosted fallback provider is billed or free-tier, for the cost-per-thousand-queries figure in the results chapter.

25. Report amendment list

Chapter by chapter, exactly what to change in the submitted report so the documents describe the same system. Make these in one pass, from this table. Rows marked **R5** are new in this revision.

REPORT LOCATION	AMENDMENT
Ch 2.5 Assumptions	Remove "data modification excluded". Replace with: an owner corrects the warehouse directly through a validated single-row path; a member proposes a correction that the owner merges.
Ch 5.1 Table 5.1	Add uploaded datasets as a data source, and note that an upload names its destination and is indexed before it is reported complete. **R5**
Ch 6.1 Table 6.1	Add FR-14 to FR-23: signup and login OTP, the owner-or-member question, invitations, access grants, file upload with destination, row correction, merge approval, clarification, follow-up, voice, export in two forms. Add FR-24 to FR-26: a member reads their own pending corrections, a merge is refused when its row moved, and a free-mail signup receives a personal workspace. **R5**
Ch 6.2 Table 6.2	Add NFR-10 account lockout, NFR-11 memory budget by configuration, NFR-12 no read path holds write privilege and every write is single-row scoped, NFR-13 rate limits on the costly paths. **R5**

REPORT LOCATION	AMENDMENT
Ch 6.4 Table 6.4	Keep viewer / analyst as database roles. Replace the three actors above them with two: owner and member. Note that an organisation is now of one of two kinds, company or personal. **R5**
Ch 8.2 Table 8.1	Pipeline is twelve steps. Step 9 resolves the caller's overlay. **R5**
Ch 8.4 Validation rules	Add the edit-path rule set: editable table, primary-key scope, exactly one row affected. Add the merge staleness rule. **R5**
Ch 9.1 Table 9.1	Add the metadata tables in §5.2, including the four merge states. Remove any glossary or staging tables.
Ch 9.2 Table 9.2	Add `mode`, `persona`, `clarified`, `used_pending`, `exported`, `confidence`. Remove `source` and `glossary_hit`. **R5**
Ch 9.4 Credential storage	Add the §9.3 registration checks — resolution, private-range refusal, TLS, privilege verification — alongside encryption at rest. **R5**
Ch 10 (new section) API contract	Reproduce §12. The report currently has no endpoint table at all, and the frontend was built against this one. **R5**
Ch 11.2 Table 11.2	Add the five-role model, the privileged path register, and object-level authorisation. **R5**
Ch 12.1 Table 12.1	Add APScheduler, Whisper, openpyxl, pandas, slowapi. Remove anything added only for the glossary.
Ch 13.1 Table 13.1	Add signup, correction, merge, staleness, overlay, object-access, rate-limit and ingestion-destination categories. **R5**
Ch 13.2 Table 13.2	Add the write-path adversarial inputs: unscoped update, non-editable table, correction by a viewer. Add a prompt-injection cell value. **R5**
Ch 14.1 Metrics	Replace the `source` filter with `mode = 'read' AND used_pending = false`, and add clarification rate as a reported metric.
Ch 15 SWOT	Strength: corrections reach the data without a spreadsheet, and every one is logged. Threat: an owner's mistyped correction is live immediately — mitigated by the edit log, the single-row scope and the required note.
Ch 16.1 Table 16.1	Add four risks: memory on the demonstration machine, a mistyped owner correction, upload parsing failure, fallback provider unavailable.
Ch 17.1 Table 17.1	Replace with the three-lane schedule in §23.
Ch 18.2 Future scope	Remove voice. Add: a review queue for owner corrections, append-mode re-upload, a support view that signup cannot reach, and calibrated confidence measured rather than assumed. **R5**
Appendix C	Add the endpoints in §12. Remove the glossary and staging endpoints. **R5**

26. Definition of done

The plan is complete when every line below is true. Lines marked **R5** are new in this revision.

- `docker compose up` starts the whole stack, models included, on a clean machine.
- All **six** privilege assertions pass in the test suite. **R5**
- Every cell of the capability matrix has a passing test.
- An unseen domain asks the owner-or-member question exactly once; a known domain never asks it; a second owner cannot be created for a domain.
- A free-mail address cannot register a **company**, and an invitation cannot be redeemed twice. **R5**
- **Two accounts created from the same free-mail provider are in different organisations, and an isolation test proves neither can read the other. R5**

- **Domain resolution never returns an organisation for a free-mail address, in any of the three `FREE_EMAIL_MODE` settings. R5**
- An owner correction changes one row and no other, and the answer on screen recalculates.
- A member correction never touches the real table, and a checksum before and after proves it.
- **A member's own answer resolves their pending value through their overlay view, the owner's identical question does not, and `used_pending` is true exactly when the rewrite fired. R5**
- **A merge whose row has changed since it was raised is refused with 409, marked stale, and shown to the owner with all three values — and the newer value survives. R5**
- A merge can be raised, approved and rejected, and the member is told which happened.
- A reminder fires while a merge request is still open.
- An update with no primary-key scope is refused, and so is one against a table with no primary key.
- An uploaded CSV becomes a queryable table in the database the uploader named, with correct types, any text fallback reported — **and is retrievable by a question immediately, with no manual reindex. R5**
- **An external connection whose host resolves into a private range, whose server refuses TLS, or whose account holds write privilege is refused at registration, naming the reason. R5**
- **A `message_id`, `thread_id` or `merge_id` from another organisation returns 403, indistinguishable from one that does not exist, with an audit row. R5**
- **An export is refused after the grant behind it is revoked. R5**
- **Every rate limit returns 429 with a retry window and still writes its `query_log` row. R5**
- **A warehouse cell containing an instruction changes nothing about the explainer's behaviour. R5**
- **Every route in §12 exists with exactly the request and response shape stated, verified by a contract test the frontend mock is also checked against. R5**
- Readout mode never returns SQL in the response body, verified by a test against the body and not the screen.
- A Readout export is every row; a Workbench export carries the charts and the analysis as well.
- Every accuracy figure in the report is computed with the §20 filter.
- OTP lockout triggers after five failed codes.
- Whisper loads on the first voice request and unloads when idle.
- `make demo` runs every step of §22 from a fresh reset.
- Every amendment in §25 is applied to the report.
- No file outside `validator.py` and `edit_validator.py` contains a safety rule.
- `.env` is absent from Git history, checked with a scan, not by memory.